

コピー伝播と死コード除去 — 順番が結果を決める

今日のゴール

O4 が残した `mv a0, s1` の連なりを消す。そして「同じパスでも、前に何を掛けたかで効果が変わる」ことを、測って確かめる。

1 この回の位置づけ

- 最適化発展シリーズの第 6 回 (2 コマ)
- 必須の前提: O2(フローグラフ)、O5(生存解析)、**O4(レジスタ割り当て)**、O1(測定)
- 編集するのは `copyprop.py` と `dce.py`

コマ	やること	到達点
1	<code>copyprop.py</code> (Step 1~3)	<code>check.py</code> が PASS。ただし命令はまだ 1 つも減らない
2	<code>dce.py</code> (Step 4~6)	4 構成の比較表が出る

2 いま残っているもの

O4 の後、`s = s + i` はこうなっている。

```

mv    a0, s2        # s を読む
addi   sp, sp, -8
sd     a0, 0(sp)
mv     a0, s1        # i を読む
ld     a1, 0(sp)
addi   sp, sp, 8
add    a0, a1, a0
sext.w s2, a0        # s へ書く

```

`add` が足しているのは、結局 `s2` と `s1` の値である。`a0` を経由する必要はない。

① O4 の直後		② コピー伝播の後		③ 死コード除去の後
<pre> mv a0, s2 addi sp, sp, -8 sd a0, 0(sp) mv a0, s1 ld a1, 0(sp) addi sp, sp, 8 add a0, a1, a0 sext.w s2, a0 </pre>		<pre> mv a0, s2 addi sp, sp, -8 sd s2, 0(sp) mv a0, s1 ld a1, 0(sp) addi sp, sp, 8 add a0, a1, s1 sext.w s2, a0 </pre>		<pre> addi sp, sp, -8 sd s2, 0(sp) ld a1, 0(sp) addi sp, sp, 8 add a0, a1, s1 sext.w s2, a0 </pre>
8 命令	コピー伝播 ⇒	まだ 8 命令。命令は減らない しかし灰色の mv はもう誰にも読まれない	死コード除去 ⇒	6 命令

コピー伝播だけでは 1 命令も減らない。死コード除去だけでも、消せる mv が無いので何も起きない。
2 つが揃って初めて効く — そしてどちらも O4(レジスタ割り当て) が mv を作った後でなければ仕事が無い

図 1 コピー伝播が死コードを作り、死コード除去が消す

3 コピー伝播

mv rD, rS の後で rD を読んでいる箇所を、rS に置き換える。

重要

コピー伝播だけでは、命令は 1 つも減らない。mv は残ったままである。減らすのは次の死コード除去の仕事で、コピー伝播はその機会を作るだけである。

コマ 1 の終わりで golden.py を走らせると効果がゼロに見えるが、それで正しい。

4 打ち切る条件

置き換えてよいのは、mv を実行したときの値がまだ有効な間だけである。途中でどちらかのレジスタが書き換えられたら、そこで打ち切る。

```

mv  a1, s1
li  s1, 0          # ← 元のレジスタが変わった
add a0, a1, a1     #   ここで a1 を s1 に置き換えたら、値が違う

```

```
mv  a1, s1
li  a1, 0      # ← 置き換え先が変わった
add a0, a1, a1  #   この a1 はもう s1 のコピーではない
```

`copy_prop_block` は、命令を 1 つ見るたびに「書き込み先が絡む表の項目を消す」ことでこれを実現する。

ブロックをまたがない。またぐには「この地点にどの定義が届くか」を知る必要があり、それは到達定義解析という別の解析になる (O5 の枠組みで書ける。発展課題)。

5 死コード除去

O5 の `live_after` を使う。書いた値が誰にも読まれない命令は消してよい。

書き込み先 $\cap$ 直後の生存集合 = $\emptyset$ $\rightarrow$ 消せる

ただし、副作用のある命令は消せない。

命令	消せるか	理由
<code>add a0, a1, a2</code>	消せる可能性	<code>a0</code> が読まれなければ不要
<code>sw a0, -24(s0)</code>	消せない	レジスタには書かないが、メモリを書き換える
<code>call f</code>	消せない	何をするか分からない
<code>beqz / j / ret</code>	消せない	制御の流れを変える

`sw` が危ないのは、生存解析から見ると「書き込み先が空集合」なので必ず死んでいるように見えるためである。特別扱いが要る。

重要

エピローグの復帰も消えてはいけない。

```
.L1:
    ld s1, -40(s0)      # s1 は「この先で読まれない」ように見える
    ld s0, 32(sp)
    ret
```

ret の use に callee-saved を入れておかないと、この ld が死コードとして消える。消えと呼び出し元の s1 が壊れる。

main だけのプログラムでは気づかない。関数を呼ぶプログラムで、呼び出し元のループ変数が壊れて無限ループになるという形で表面化する。05 の RETURN_USES がここで効いてくる。

6 繰り返す理由

1 つ消すと、その命令が読んでいた値が誰にも読まれなくなり、別の命令が新たに死ぬ。

```
mv    a1, s1           # 2 周目で死ぬ
add   a0, a1, a1        # 1 周目で死ぬ (a0 が後で上書きされるとき)
li    a0, 7
```

だから、消せる命令が無くなるまで繰り返す。

7 実装

Step	ファイル	実装対象
1	copyprop.py	replace_uses (読んでいる側だけ置き換える)
2	copyprop.py	copy_prop_block (打ち切り条件が肝)
3	copyprop.py	run
4	dce.py	has_side_effect
5	dce.py	is_dead
6	dce.py	run (消えなくなるまで繰り返す)

```
B=../01_measure/bench/loop_sum.c
python3 ../optcc.py --regalloc --passes isel,copyprop,dce $B
python3 ../optcc.py --passes isel,copyprop,dce $B
```

2 つ目 (レジスタ割り当てなし) がほとんど何も減らないことを、自分で確かめてほしい。

8 測ってみると

まず、04 の後にこの回をかけた効果。

ベンチマーク	静的:04 後	+06	削減	動的:04 後	+06	削減
bubble	249	228	8.4%	18809	16940	9.9%
fib_rec	74	73	1.4%	69062	69061	0.0%
loop_sum	49	44	10.2%	5032	4230	15.9%
matmul	299	271	9.4%	21544	19346	10.2%
strops	189	173	8.5%	59817	51815	13.4%
合計	860	789	8.3%	174264	161392	7.4%

そして、この回の主題である順番の効果。

構成	静的	削減	動的	削減	
isel だけ (基準)	997	0.0%	190278	0.0%	
isel,copyprop,dce	996	0.1%	190277	0.0%	← 割り当てなし
regalloc + isel	860	13.7%	174264	8.4%	
regalloc + 全部	789	20.9%	161392	15.2%	

まったく同じ **copyprop,dce** が、**997 命令中 1 命令**しか減らせない場合と、**71 命令**減らせる場合がある。違いは「前にレジスタ割り当てを掛けたかどうか」だけである。

レジスタ割り当ての前は、値がすべてスタックを経由するので **mv** がほとんど出てこない。消す対象が無いのだから、どんなに正確な解析をしても結果は変わらない。

04 が **06** の仕事を作っている。これを **phase ordering** という。

9 発展課題

1. 順番を入れ替える: `dce, copyprop` の順にすると結果は変わるか。 `copyprop, dce, copyprop, dce` と 2 回繰り返すとどうなるか
2. ストア転送: `sd a0, 0(sp)` の直後に `ld a1, 0(sp)` があれば、 `mv a1, a0` にできる。すると `push/pop` が死ぬ。B2(レジスタスタック) と同じ効果が、解析から導ける
3. 到達定義解析: ブロックをまたぐコピー伝播に必要な、前向きの解析。O5 の枠組みで書ける。方程式はどうなるか
4. `fib_rec` が 0.0% の理由: O4 が昇格しなかったので `mv` が出ていない。O4 の `worth_promoting` を無効にして測ると、この回の効果はどう変わるか
5. どこまで減るか: `regalloc + isel, copyprop, dce, layout` を測る。残った命令のうち、まだ無駄に見えるものはあるか

補足

コラム: 最適化の順番には「正解」が無い。A を先かけると B の機会が増え、B を先かけると A の機会が増える —— という関係はよくある。どちらを先にするかで結果が変わり、しかもプログラムによって最適な順番が違う。

実在のコンパイラは、経験的に決めた順番でパスを並べ、同じパスを何度も繰り返しかける (LLVM の `-O2` は数十個のパスを決まった順に走らせる)。「最適な順番を求める」問題自体が難しいので、実務では測って決める。この回でやったことが、まさにそれである。